

A Predictive Emissions Monitoring System: Comparing and Compressing Machine Learning Models for Gas Turbine NOx Prediction

MSc People-Centred Artificial Intelligence (Online)

EEEM075, AI and Sustainability

Coursework Assignment

Student URN: 6953300

Contents

Executive summary3

1. Introduction4

1.1 Background and problem statement, 1.2 Objectives and hypotheses, 1.3 Data overview, 1.4 Literature review, 1.5 Relevance to the UN Sustainable Development Goals, 1.6 Approach overview4

2. Data understanding, pre-processing, and exploration7

2.1 Loading and structure, 2.2 Data cleaning, 2.3 Descriptive statistics, 2.4 Exploratory analysis, 2.5 Feature scaling, 2.6 Train/validation/test split, 2.7 Feature engineering7

3. Modelling12

3.1 Problem formulation, 3.2 Model selection, 3.3 Hyperparameter tuning, 3.4 Implementation and training procedure, 3.5 Baseline vs. tuned results12

4. Performance evaluation14

4.1 Evaluation methodology, 4.2 Test set results, 4.3 Computational cost comparison, 4.4 Explainable AI (SHAP), 4.5 Investigating the drop in test performance, 4.6 Validity check: separating model quality from concept drift, 4.7 Energy and lifecycle-efficiency implications14

5. Model compression19

5.1 MLP: structured pruning and quantisation, 5.2 XGBoost: ensemble truncation and distillation, 5.3 Results, 5.4 Discussion of the trade-offs19

6. Discussion, conclusion, and recommendations21

6.1 Interpretation of findings and sustainability implications, 6.2 Limitations, 6.3 Conclusions, 6.4 Recommendations for future work21

References24

Appendix A, Full train vs. validation feature-range comparison25

Appendix B, Selected hyperparameter configurations25

List of figures

Figure 1: Distributions of NOX, CO, TIT and TEY (training set), Figure 2: Boxplots of all sensors and targets, Figure 3: Correlation matrix (training set), Figure 4: Mean CO and NOX by year, Figure 5: MLP training and validation loss curves, Figure 6: Mean absolute SHAP values, XGBoost, Figure 7: Mean absolute SHAP values, MLP, Figure 8: Accuracy, size and latency across all model variants

List of tables

Table 1: Data dictionary, Table 2: Descriptive statistics, Table 3: Engineered-feature selection, Table 4: Baseline vs. tuned validation performance, Table 5: Test set performance, Table 6: Model size, latency and training time, Table 7: Top SHAP-ranked features, Table 8: Train vs. validation feature ranges (summary), Table 9: Accuracy under three evaluation protocols, Table 10: Per-year error breakdown, Table 11: Compression results

Executive summary

This report builds and evaluates a Predictive Emissions Monitoring System (PEMS), a machine-learning "soft sensor" that infers gas turbine NO_x emissions from nine cheap, already-available ambient and operational sensor readings, as a lower-cost, software-only alternative to physical Continuous Emissions Monitoring System (CEMS) hardware. The project uses five years (2011–2015) of hourly-aggregated turbine data from the UCI Gas Turbine CO and NO_x Emission Data Set, with a strict chronological split (2011–2013 train, 2014 validation, 2015 held-out test) chosen to avoid the information leakage a random split would introduce in time-ordered sensor data.

Two models were trained from scratch and compared on equal footing: a multi-layer perceptron (MLP) and an XGBoost gradient-boosted ensemble. After a validation-based hyperparameter search, the tuned MLP reached a moderate, genuinely useful validation R^2 of 0.322, while XGBoost's validation R^2 remained negative throughout tuning. On the held-out 2015 test set, however, both models degraded substantially (MLP R^2 -0.103, XGBoost R^2 -0.643). This drop was investigated directly rather than reported as a bare number: a feature-range comparison between the training and validation years ruled out covariate shift (fewer than 0.1% of validation rows fell outside the training feature ranges), pointing instead to **concept drift**, an unrecorded change in the sensor-to-NO_x relationship itself, most likely a maintenance event, recalibration, or control adjustment not captured by any of the nine recorded variables. A dedicated validity check confirmed this diagnosis from the other direction: on an in-distribution holdout drawn from the training years, both models are strong (MLP R^2 0.747; XGBoost R^2 0.879), demonstrating that the pipeline and models are sound and that the chronological failure is a property of the data, not the modelling. This finding, not a headline accuracy figure, is treated as the project's central and most useful result, because a model that degrades silently without a way to detect that degradation is not a trustworthy substitute for physical monitoring.

SHAP explainability was applied to both models using a consistent, model-agnostic method. Both independently ranked ambient temperature as the single most influential feature and placed turbine inlet temperature in their top three despite its weak linear correlation with NO_x, evidence that both models learned a physically sensible, non-linear relationship consistent with thermal-NO_x theory, rather than fitting noise.

Both models were then compressed using two techniques suited to their respective architectures: the MLP via structured pruning (fine-tuned afterwards) and post-training dynamic quantisation; XGBoost via ensemble truncation and knowledge distillation into a much smaller student model. Quantisation gave a 63% size reduction with accuracy marginally *better* than baseline, a close-to-free efficiency gain. More strikingly, every XGBoost compression variant outperformed its full-size original on the test set, with the 34.5 KB distilled student (98% smaller than the 1.78 MB baseline) also the most accurate XGBoost variant, evidence that compression acted as an additional regulariser against the concept-drift problem identified earlier, not merely a size/accuracy trade-off.

Overall, the MLP is the stronger model on test accuracy, disk size, and inference latency, while XGBoost trains an order of magnitude faster and is clearly the stronger model within a stable data period, a more nuanced picture than either model simply "winning", and one that runs counter to the project's initial expectation that the tree ensemble would be the more efficient of the two. The project's conclusions and recommendations (Section 6) centre on the need for drift detection and periodic retraining in any real deployment, rather than treating a once-trained model as permanently valid.

1. Introduction

1.1 Background and problem statement

Gas turbines are widely used across power generation and oil & gas (power islands, mechanical drive for compressors), and their combustion stage produces two regulated pollutants: carbon monoxide (CO) and nitrogen oxides (NO_x). The two sit in tension, NO_x formation is dominated by the thermal (Zeldovich) pathway and scales with flame temperature, so it tends to rise with load, while CO is a marker of incomplete combustion and tends to rise at low load and poor mixing. A single operating point cannot minimise both simultaneously, which makes emissions management a genuine, ongoing optimisation problem rather than a solved one.

Plants are required to monitor stack emissions continuously. The conventional approach is a Continuous Emissions Monitoring System (CEMS): physical hardware that is accurate but expensive, maintenance-heavy, and prone to drift and downtime. An established alternative is a Predictive Emissions Monitoring System (PEMS), a software model that infers emissions from process variables the plant already measures, acting as a soft sensor (Kaya, Tüfekci and Uzun, 2019). This coursework builds and evaluates such a soft sensor.

Problem statement. Can a supervised model learn a mapping from cheap, already-available ambient and operational sensor readings to turbine emissions, accurately enough to act as a virtual emissions sensor, and can that model be made lightweight enough to be deployable and energy-efficient in its own right?

Assumptions. We assume the five years of recorded process variables are sufficient to explain the majority of variation in emissions, acknowledging (see Section 2) that some year-on-year drift in mean emissions is not fully explained by the nine recorded sensors, plausibly due to unrecorded changes in plant operation or maintenance. We also assume that hourly-aggregated readings are an adequate resolution for this task, consistent with how the data was collected and published.

1.2 Objectives and hypotheses

- **O1:** Quantify how accurately NO_x (primary target) can be predicted from nine ambient/operational sensor readings, using two different supervised approaches, with CO identified as a secondary target for extension.
- **O2:** Compare a neural network (MLP) against a tree-based ensemble (XGBoost) on both predictive accuracy and computational cost (training time, inference latency, model size), establishing which is the more efficient baseline.
- **O3:** Apply explainable AI (SHAP) to check whether the models' predictions are consistent with known combustion physics (e.g. load driving CO, flame temperature driving NO_x), rather than treating either model as a black box.
- **O4:** Compress both trained models using at least two techniques each, and quantify the accuracy/size/speed trade-off this introduces, treating compression as central to the project's argument, not an afterthought.

Two working hypotheses were stated before modelling began. **H1:** the nine process sensors carry enough information to predict NO_x with useful accuracy (validation R² meaningfully above zero). **H2:** the tree ensemble would prove the more computationally efficient of the two models at inference time. Both are revisited explicitly in the results: H1 is supported within a stable data period but qualified heavily by concept drift (Sections 4.5–4.6), and H2 was falsified by direct measurement (Section 4.3).

1.3 Data overview

The dataset is the UCI Gas Turbine CO and NOx Emission Data Set (Kaya, Tüfekci and Uzun, 2019; UCI Machine Learning Repository, 2019), collected from a gas turbine in north-western Turkey between 01.01.2011 and 31.12.2015. It contains 36,733 hourly-aggregated records across 11 continuous sensor variables (nine input features plus the two emissions targets), with no missing values. The data is provided as five yearly files and is already in chronological order.

The dataset is complex enough to pose a genuine challenge: it is large (36,733 rows), entirely continuous-valued, spans a wide operating envelope (ambient temperatures from -6°C to $+37^{\circ}\text{C}$; turbine loads from 100 to 180 MWH), and, most importantly, covers five calendar years of real plant operation, which exposes the models to slow, unrecorded changes in the physical system itself. As Sections 4.5–4.6 show, this temporal dimension turned out to be the hardest and most instructive property of the data.

Table 1: Data dictionary for the Gas Turbine CO and NOx Emission Data Set, nine ambient/operational input features and two emissions targets, all continuous, hourly-aggregated.

Variable	Description	Unit	Role
AT	Ambient temperature	$^{\circ}\text{C}$	Feature
AP	Ambient pressure	mbar	Feature
AH	Ambient humidity	%	Feature
AFDP	Air filter difference pressure	mbar	Feature
GTEP	Gas turbine exhaust pressure	mbar	Feature
TIT	Turbine inlet temperature	$^{\circ}\text{C}$	Feature
TAT	Turbine after temperature	$^{\circ}\text{C}$	Feature
TEY	Turbine energy yield	MWH	Feature
CDP	Compressor discharge pressure	mbar	Feature
CO	Carbon monoxide	mg/m^3	Target (secondary)
NOX	Nitrogen oxides	mg/m^3	Target (primary)

The dataset's original benchmark protocol recommends training/cross-validating on the first three years and testing on the last two (Kaya, Tüfekci and Uzun, 2019). We follow this, with one refinement: 2011–2013 for training, 2014 as a validation set, and 2015 held out entirely as the test set, giving a genuine, leakage-free generalisation check in addition to the original protocol's structure.

An initial data quality pass (see Section 2) found the data otherwise clean but identified 478 records (1.3%) with ambient humidity readings above 100%, a physically impossible value for a relative measure, handled as a sensor artefact rather than treated as informative.

1.4 Literature review

Predictive Emissions Monitoring Systems are an active area of applied research, not a niche idea: a recent review synthesising current PEMS technology trends confirms that machine learning-based PEMS can reduce capital costs by roughly 50% and operational costs by roughly 90% relative to hardware CEMS, while highlighting LSTM, temporal convolutional networks, and stacked model architectures as the current state of the art for accuracy (Wang et al., 2025). This motivates the project's premise directly, though this project deliberately uses simpler, more transparent architectures (an MLP and XGBoost) given the module's emphasis on explainability and compression alongside raw accuracy. Closer to the specific dataset used here, Pachauri

(2024) applies ensemble machine learning methods to the same class of gas turbine CO/NO_x prediction problem, reinforcing that tree-based ensembles are a reasonable, competitive choice of model for this task family, which is part of the justification for including XGBoost as one of the two models compared in Section 3, even though it ultimately proved the weaker performer here.

The explainability approach draws on the SHAP framework of Lundberg and Lee (2017), applied with the caution recommended by Kantz et al. (2024), who specifically stress-test SHAP and related XAI methods in an industrial process modelling context (an electric arc furnace) and find that explanation robustness varies meaningfully by method and by how noisy the underlying process is, a caution applied in Section 4, where SHAP results are cross-checked against known combustion physics rather than accepted at face value.

The project's central empirical finding, that both models degrade from validation to test year in a way traced to concept drift rather than covariate shift, sits within a broader, active literature on drift in deployed learning systems (Gama et al., 2014). Zink, Ioshchikhin and Weigold (2024) study concept drift detection specifically for industrial load forecasting with neural networks, confirming that machine learning models supervising industrial processes routinely need explicit drift-monitoring mechanisms because they "frequently are not able to generalise to changes in data caused by environmental changes or degrading sensors", precisely the failure mode diagnosed in Section 4.5, and the basis for the future-work recommendation in Section 6.4.

The compression techniques applied in Section 5 are grounded in established methods rather than improvised. Knowledge distillation follows the foundational formulation of Hinton, Vinyals and Dean (2015), training a smaller student model against a larger teacher's outputs. The broader motivation for compression, that energy-efficient deep learning is achievable at multiple points across a model's lifecycle, including post-training compression, follows the lifecycle framing of Mehlh, Schacht and Lanquillon (2023), and the principle that computational cost should be reported alongside accuracy as a first-class evaluation criterion follows Schwartz et al. (2020). Finally, Toderas (2025) provides a systematic review of AI's role in advancing the Sustainable Development Goals while explicitly flagging AI's own ecological footprint as a critical, underexamined risk, directly supporting this project's decision (Section 1.5) to treat the *sustainability of the AI system itself* as a first-class objective (O4) rather than an afterthought bolted onto an accuracy-focused project.

1.5 Relevance to the UN Sustainable Development Goals

This project sits at the intersection of two framings the module distinguishes: *AI for sustainability* and *sustainability of AI*. A cheap, always-on PEMS gives plant operators continuous visibility into emissions without physical CEMS hardware, supporting lower-emission operation and better-informed control decisions. Within the 2030 Agenda (United Nations, 2015), this links directly to **SDG 7** (Affordable and Clean Energy) and **SDG 13** (Climate Action), and more broadly to **SDG 9** (Industry, Innovation and Infrastructure, resilient, monitorable industrial process) and **SDG 3** (Good Health and Well-being, via air quality). It is worth being precise about the mechanism: the model predicts emissions for monitoring purposes; emissions reduction is a downstream benefit enabled by cheaper, continuous visibility, not something the model performs directly. Separately, the project treats the sustainability of the AI system itself as part of the goal, not a footnote: a monitoring tool that is itself computationally wasteful undermines its own premise, which is the direct motivation for the compression stage in Section 5.

1.6 Approach overview

1. Load, quality-check, and chronologically split the five years of data (Notebook 1).

2. Explore feature distributions, correlations against combustion theory, and time trends (Notebook 2).
3. Test two physics-motivated engineered features against a fixed decision rule, then train two models, an MLP and XGBoost, on the primary target (NO_x), first as baselines, then with hyperparameter tuning (Notebook 3).
4. Evaluate both models on the held-out test set using regression metrics (RMSE, MAE, R²), compare computational cost, and apply SHAP explainability (Notebook 4).
5. Compress both models using two techniques each and re-evaluate on the same metrics (Notebook 5).
6. Run a dedicated validity check separating model quality from concept drift: an in-distribution holdout, a random-split sensitivity run, and a per-year error breakdown (Notebook 6).

2. Data understanding, pre-processing, and exploration

2.1 Loading and structure

The five yearly files (gt_2011.csv–gt_2015.csv) were loaded separately, each tagged with its year, then concatenated into a single table of 36,733 hourly records across 11 sensor/target columns (Table 1). No file required format correction, column names and units were consistent across all five years, consistent with the dataset being a single continuous instrument record rather than a merger of heterogeneous sources.

2.2 Data cleaning

Three checks were run before any statistical treatment of the data: missing values, physically impossible readings, and statistical outliers, treated as three separate questions, since each implies a different response.

Missing values. None were found in any column across any of the five years.

Physically impossible readings. Ambient humidity (AH) is a relative measure and cannot physically exceed 100%. 478 records (1.3% of the dataset) had AH above 100%, a genuine impossibility, not just the high end of a real distribution, so it was treated as a sensor/calibration artefact and capped at 100 rather than dropped. Dropping would leave gaps in what is otherwise a continuous, time-ordered record, and 1.3% of readings is too large a slice to discard outright without a specific reason to distrust the surrounding data. No negative CO or NO_x readings were found, so no equivalent correction was needed there.

Statistical outliers. An IQR-based check flagged a non-trivial share of values in several columns, most notably TAT (13.5% of rows) and CO (7.2%). These were deliberately *not* removed. A gas turbine operates across a wide load range, and both TAT (turbine after temperature) and CO are known to vary sharply with load, the EDA in Section 2.4 confirms this is exactly what is happening, rather than measurement error (see the boxplots in Figure 2). Applying a blanket IQR filter here would have removed genuine, informative operating conditions rather than noise, which would have quietly simplified the prediction problem in a way that doesn't reflect the real deployment setting.

2.3 Descriptive statistics

Table 2: Descriptive statistics for all sensor and target variables (raw values before the ambient-humidity cap, all five years, n=36,733).

Variable	Mean	Std	Min	Median	Max
AT (°C)	17.71	7.45	-6.23	17.80	37.10
AP (mbar)	1013.07	6.46	985.85	1012.60	1036.60
AH (%)	77.87	14.46	24.08	80.47	100.20
AFDP (mbar)	3.93	0.77	2.09	3.94	7.61
GTEP (mbar)	25.56	4.20	17.70	25.10	40.72
TIT (°C)	1081.43	17.54	1000.80	1085.90	1100.90
TAT (°C)	546.16	6.84	511.04	549.88	550.61
TEY (MWH)	133.51	15.62	100.02	133.73	179.50
CDP (mbar)	12.06	1.09	9.85	11.96	15.16
CO (mg/m ³)	2.37	2.26	0.00	1.71	44.10
NOX (mg/m ³)	65.29	11.68	25.90	63.85	119.91

These figures are computed on the raw, uncleaned data, before the ambient-humidity cap described in Section 2.2, this is why AH's max (100.20) sits fractionally above the physical ceiling of 100%, consistent with the 478 affected records identified there. A few other patterns are worth noting ahead of the correlation analysis below: TAT's mean (546.16°C) sits close to its max (550.61°C) with a comparatively small standard deviation, suggesting the turbine spends much of its time near an upper operating ceiling for this variable rather than varying freely across its full range; CO's standard deviation (2.26) is nearly as large as its mean (2.37), and its max (44.10) is roughly 19 times its mean, a strong early signal of the right-skewed, load-dependent distribution confirmed in Section 2.4 below, and the reason CO was not treated as a straightforward candidate for outlier removal in Section 2.2.

2.4 Exploratory analysis

Feature distributions were inspected for all nine input sensors and the two targets (Figure 1). Several (notably CO, TAT, TEY) show multi-modal or heavily skewed shapes rather than a single smooth peak, consistent with a turbine spending time in distinct operating regimes (different load levels) rather than hovering around one steady state. TAT in particular shows a sharp left-skewed shape with a dense cluster near its ceiling (consistent with the descriptive statistics above), while TEY shows visibly separated modes, plausibly corresponding to a small number of common dispatch/load setpoints rather than a continuum of operating levels. CO's distribution is heavily right-skewed with a long tail, the majority of hours sit at low CO, with a comparatively small number of high-CO hours pulling the mean well above the median (2.37 vs. 1.71, Table 2), which is itself consistent with CO spiking specifically during low-load or transient operation rather than varying smoothly with time.

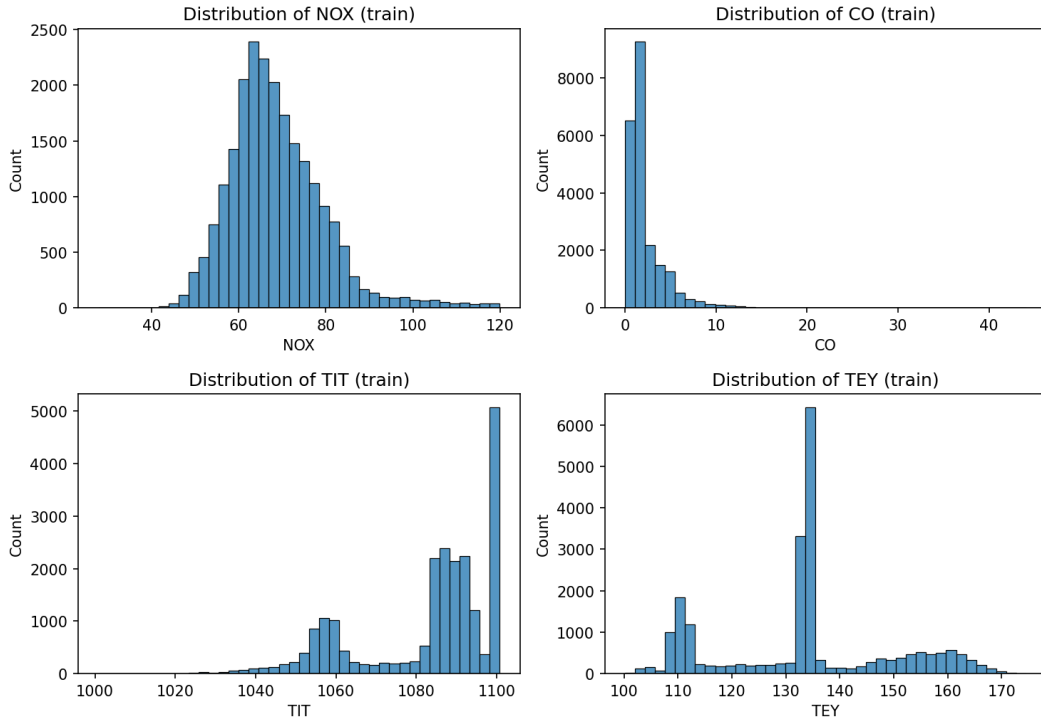


Figure 1: Distributions of the two emissions targets (NOX, CO) and two load-related sensors (TIT, TEY) on the training years (2011–2013). CO is heavily right-skewed and TEY multi-modal, consistent with distinct load regimes rather than a single steady operating state.

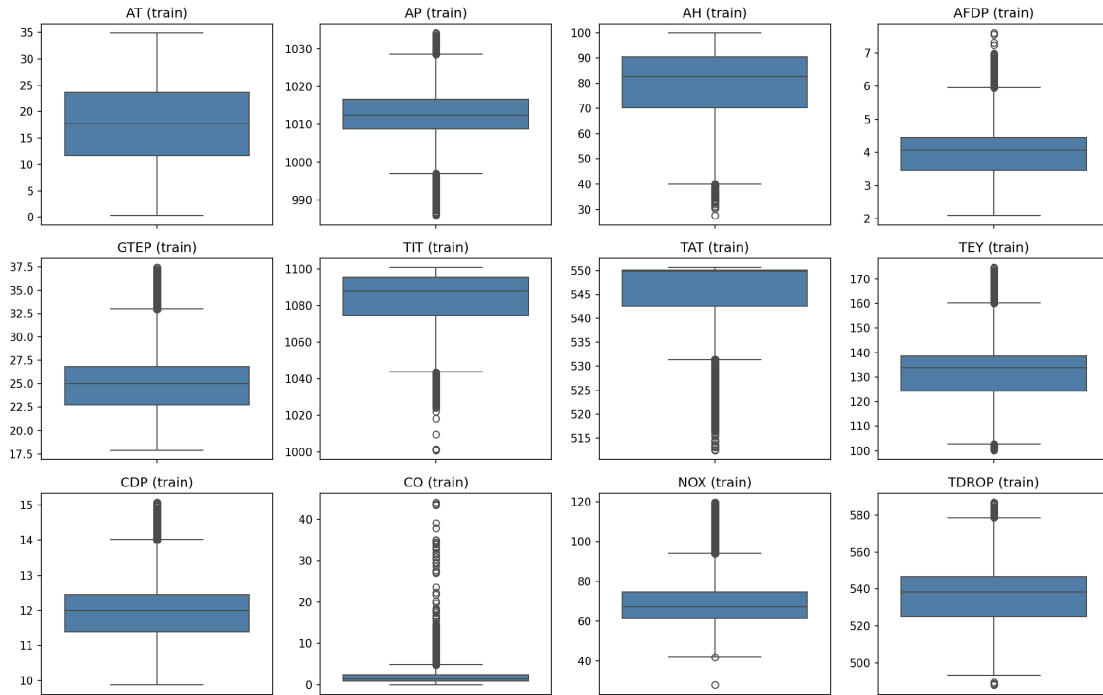


Figure 2: Boxplots of all nine sensor features and both targets (training years). The IQR-flagged values discussed in Section 2.2 concentrate in TAT (readings pinned near its 550°C operating ceiling) and CO (high-emission low-load hours); both were retained as genuine operating conditions rather than removed as outliers.

A correlation matrix (computed on the training years only, to avoid letting validation/test characteristics influence any modelling decision) was checked against basic combustion theory before any model was built (Figure 3):

- CO correlates strongly negatively with the load-related sensors, TIT (-0.73), TEY (-0.59), CDP (-0.57), GTEP (-0.55). This matches expectation directly: incomplete combustion, and therefore more CO, occurs at low load.

- NOx correlates strongly negatively with AT, ambient temperature (-0.59), but only weakly with TIT (-0.20). This second point is worth flagging explicitly: textbook thermal-NOx theory says flame temperature (TIT) should be a dominant driver, so a weak linear correlation is not the same as a weak relationship, Section 4.4 revisits this directly with SHAP, once trained models (which can capture non-linear effects) are available to check.

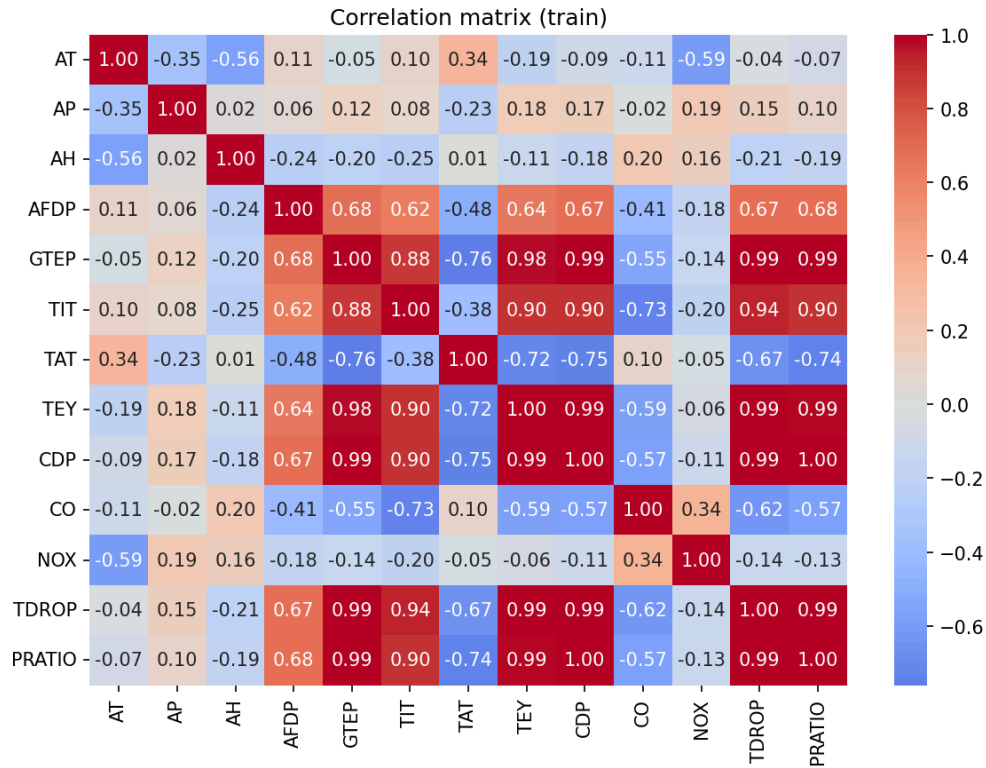


Figure 3: Correlation matrix of all sensor variables and both targets (training years only). CO tracks the load-related sensors negatively (TIT -0.73, TEY -0.59), while NOx correlates most strongly with ambient temperature (AT, -0.59) and only weakly with TIT (-0.20), the linear-correlation puzzle revisited with SHAP in Section 4.4.

The year-by-year trend check (Figure 4) surfaced the single most consequential property of this dataset: mean NOx drops from roughly 68–70 mg/m³ in 2011–2013 to roughly 60 mg/m³ in 2014–2015, with no corresponding drop in CO and no matching shift in any input feature's range (Section 4.5). At the EDA stage this was recorded as a caution for modelling; by Section 4 it becomes the central finding of the project.

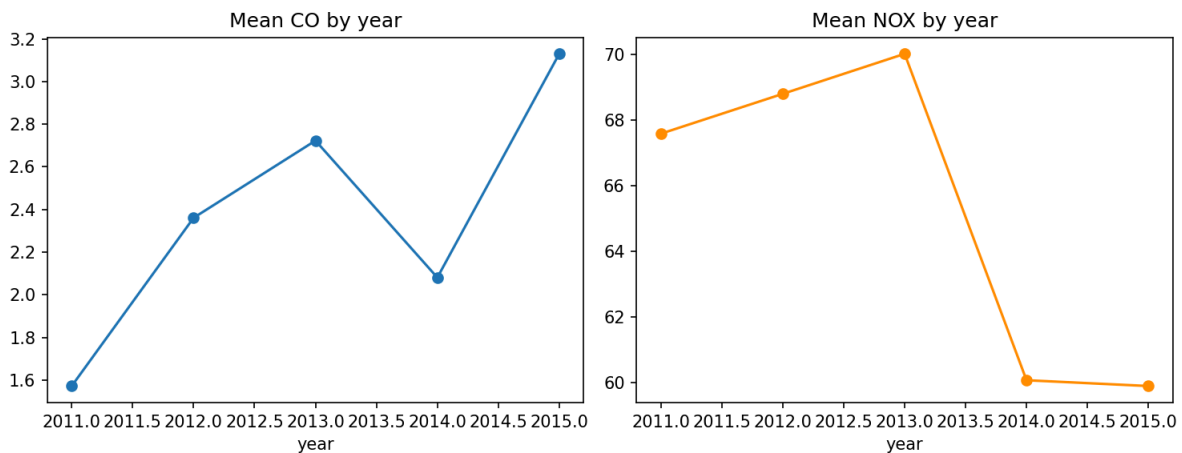


Figure 4: Mean CO and NOx by year, 2011–2015. Mean NOx falls from ≈68–70 mg/m³ (2011–2013) to ≈60 mg/m³ (2014–2015) with no corresponding fall in CO, the first indication of the target drift investigated in Sections 4.5–4.6.

2.5 Feature scaling

Scaling (standardisation to zero mean and unit variance, fitted on the training years only) was applied for the MLP but deliberately **not** for XGBoost. This is not an oversight: gradient-based optimisation (used to train the MLP) is sensitive to the relative scale of input features, since large-magnitude features can dominate the gradient early in training; tree-based splits, by contrast, threshold each feature independently and are invariant to monotonic rescaling. Applying an unnecessary transformation to XGBoost's inputs would add a processing step with no benefit, using the appropriate technique for the model is as relevant to pre-processing as it is later to compression.

2.6 Train / validation / test split

The data was split **chronologically, not randomly**: 2011–2013 for training (22,191 rows), 2014 for validation (7,158 rows), and 2015 held out entirely as the test set (7,384 rows), untouched until Section 4. A random shuffle-based split was deliberately avoided, since the data is a time-ordered sequence of hourly readings and a random split would let information from later time points leak into training, an unrealistic setup, since a real deployment could only ever train on the past to predict the future. This also matches the benchmark protocol suggested in the dataset's own documentation (train on the first three years, test on the last two), extended here with an explicit validation year for hyperparameter selection rather than tuning directly against the test set. Section 4.6 quantifies exactly how much difference this protocol choice makes.

2.7 Feature engineering

Two engineered features were constructed, each with a physical rationale rather than blind combinatorics: **TDROP** = TIT - TAT, the temperature drop across the turbine, a direct proxy for the work extracted from the gas path, capturing energy-conversion intensity in a single variable; and **PRATIO** = CDP / AP, the overall pressure ratio (compressor discharge normalised by ambient pressure), a dimensionless operating-point descriptor that removes day-to-day weather variation in ambient pressure from the load signal.

Whether they earned their place was decided empirically, with the decision rule fixed *before* looking at the results: both baseline models were trained twice, once on the nine raw sensors, once with the two engineered features added, and the engineered set would be kept only if it reduced the average validation RMSE across the two models. It did not (Table 3): both models performed slightly worse with the added features, indicating the base sensors already carry the information the engineered combinations encode (unsurprising in hindsight, given TDROP and PRATIO are deterministic functions of existing inputs, they add convenience, not information, and in this case added mild redundancy instead). The nine base sensors were therefore retained for all subsequent modelling, and the decision was recorded programmatically in the pipeline (artifacts/features.json) so later notebooks inherit it automatically.

Table 3: Engineered-feature selection on the validation set, baseline models trained with and without TDROP and PRATIO. The engineered features increased validation RMSE for both models and were rejected under the pre-registered decision rule.

Feature set	MLP RMSE	XGBoost RMSE	Average RMSE
Base (9 sensors)	9.42	10.49	9.96
Base + engineered (11)	9.65	10.65	10.15

3. Modelling

3.1 Problem formulation

The task is framed as supervised regression: predict NO_x (the primary target) from the nine ambient/operational sensors (AT, AP, AH, AFDP, GTEP, TIT, TAT, TEY, CDP). CO is deliberately excluded from the input features, even though it is present in the same files. Using one pollutant reading to help predict another would defeat the purpose of a soft sensor whose entire value proposition is inferring emissions from cheap, already-available process sensors, not from other emissions hardware the plant is trying to replace in the first place.

3.2 Model selection

Two models were trained from scratch, as required, chosen to represent genuinely different learning paradigms rather than two minor variants of the same idea:

Model 1, MLP (multi-layer perceptron, PyTorch). A small feed-forward neural network. Strengths: learns smooth, continuous, potentially non-linear mappings; naturally suited to gradient-based fine-tuning, which later matters directly for the pruning technique in Section 5. Weaknesses: needs feature scaling, more hyperparameters to tune, less immediately interpretable without a dedicated technique such as SHAP.

Model 2, XGBoost (gradient-boosted trees; Chen and Guestrin, 2016). Strengths: handles non-linearities and feature interactions natively without scaling, generally strong on tabular data with comparatively little tuning, and its axis-aligned splits are easy to reason about structurally. Weaknesses, which became directly relevant to this project's results (Section 4): a boosted ensemble's predictions are effectively a lookup over the exact regions of feature space seen during training, which makes it harder for the model to remain reasonable once the underlying relationship shifts away from the training period.

3.3 Hyperparameter tuning

Method. A validation-based random search was used for both models, not k-fold cross-validation. Cross-validation shuffles rows into folds, which would let training-period data sit in the same fold as future validation-period data, an information leak for time-ordered readings. Instead, every candidate configuration was trained once on 2011–2013 and scored once against the fixed 2014 validation set, with the lowest validation RMSE selecting the winner. The test set (2015) played no role in this process.

Search space. XGBoost (20 random trials): `max_depth` in {2,3,4,5,6}, `learning_rate` in {0.01, 0.03, 0.05, 0.1}, `n_estimators` in {100, 200, 300, 500}, `subsample` in {0.6, 0.8, 1.0}, `colsample_bytree` in {0.6, 0.8, 1.0}, `reg_alpha` in {0, 0.1, 1.0}, `reg_lambda` in {1.0, 5.0, 10.0}. Shallower trees and stronger regularisation were deliberately included in the search range, in response to the untuned baseline's poor validation performance (Section 3.5). MLP (6 hand-picked configurations): architectures ranging from a single 16-unit hidden layer up to three hidden layers (128-64-32), dropout from 0.0–0.3 (Srivastava et al., 2014), learning rate from 5e-4 to 1e-3, weight decay from 0 to 1e-3, spanning from a small, heavily regularised network to a larger, more expressive one, to establish whether the baseline's weakness was a capacity problem or a regularisation problem.

Trial counts were kept modest given the project's CPU-only, part-time schedule, a limitation acknowledged here rather than hidden, and revisited in Section 6. The winning configurations are stated in full in Appendix B.

3.4 Implementation and training procedure

Both models were trained with a fixed random seed (42, applied to NumPy, PyTorch, and Python's random module) so that repeated runs of the same configuration are reproducible.

MLP. Trained with the Adam optimiser and mean-squared-error loss, batch size 256, for up to 50 epochs. Training used early stopping on validation RMSE with a patience of 8 epochs, if validation performance failed to improve for 8 consecutive epochs, training stopped and the best-performing weights were restored, rather than training for a fixed epoch count regardless of overfitting. This is the same mechanism, at a shorter patience (5 epochs) and lower learning rate, later reused for the fine-tuning step after structured pruning in Section 5.1.

XGBoost. Trained via the scikit-learn-compatible XGBRegressor interface with the default squared-error objective, using each candidate's full hyperparameter set (Section 3.3) rather than an internal early-stopping round, since the validation-based random search already selects `n_estimators` externally as one of the tuned hyperparameters.

Both models were trained only on the 2011–2013 training split; the 2014 validation set was used exclusively to score candidates during tuning and was never used to update either model's weights directly.

3.5 Baseline vs. tuned results (validation set, 2014)

Table 4: Baseline and tuned validation performance (2014), with single-fit training time measured on the same CPU-only machine. Tuning nearly tripled the MLP's R^2 while XGBoost's remained negative throughout.

Model	RMSE	MAE	R^2	Training time (s)
MLP (baseline)	9.42	7.34	0.107	39.2
MLP (tuned)	8.21	6.15	0.322	35.5
XGBoost (baseline)	10.49	9.13	-0.107	2.2
XGBoost (tuned)	10.35	9.01	-0.076	2.6

Tuning produced a substantial, genuine improvement for the MLP, R^2 nearly tripled (0.107 to 0.322), indicating the untuned baseline had simply been under-regularised rather than fundamentally unsuited to the task. The selected configuration is the largest, most heavily regularised one in the search (three hidden layers, dropout 0.3; Appendix B), and its training diagnostics show the expected healthy pattern: training and validation loss fall together before validation loss plateaus and early stopping halts training (Figure 5), indicating the final model is neither badly underfit nor left to overfit the training years.

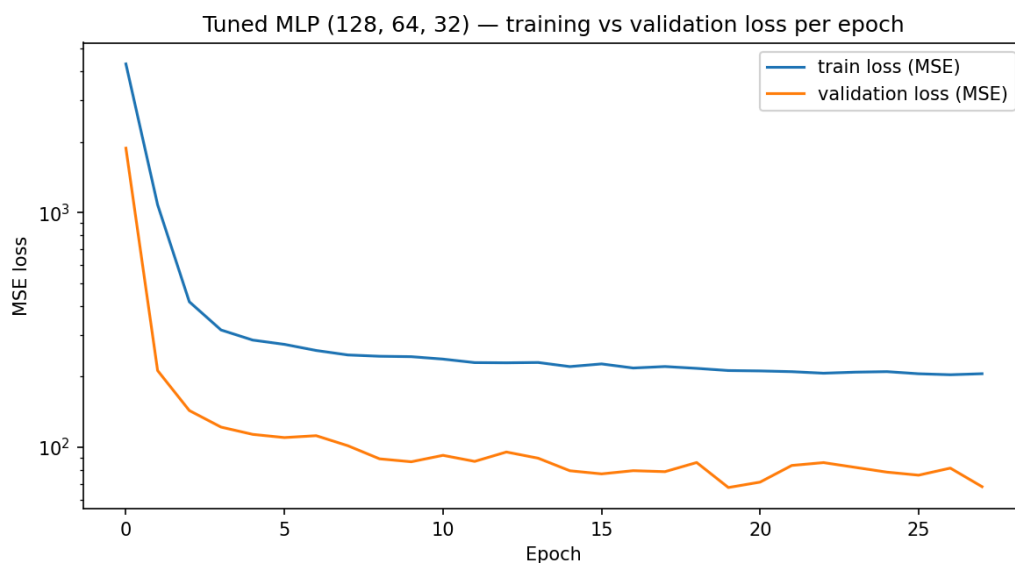


Figure 5: Training and validation loss (MSE) by epoch for the tuned MLP configuration. Validation loss plateaus while training loss continues to fall slowly; early stopping (patience 8) halts training at the plateau and restores the best weights, containing overfitting to the training years.

XGBoost improved only marginally despite an equally thorough search, and its validation R^2 remained negative throughout, worse than predicting the mean NOx value for every row. Notably, its training-time advantage is enormous (roughly 14× faster per fit, Table 4), which matters later in the lifecycle analysis (Section 4.7). This gap between the two models' response to tuning motivated a direct investigation, reported in Sections 4.5–4.6, into *why* one model responded to tuning and the other effectively could not.

4. Performance evaluation

4.1 Evaluation methodology

Three regression metrics were used throughout, chosen for complementary reasons: **RMSE** and **MAE** (both in the original NOx units, mg/m³) give directly interpretable measures of typical prediction error, with RMSE weighting large errors more heavily; **R^2** benchmarks each model against the simplest possible baseline, predicting the mean NOx value for every row, which is what makes it possible to say a model is not just imprecise but actually *worse than not modelling at all* when R^2 is negative, as happened here. All three were computed identically for every model and every variant (baseline, tuned, compressed) so that results are directly comparable throughout the report. Alongside accuracy, every model variant is also measured on disk size, inference latency, and training time, treating computational efficiency as a first-class evaluation criterion rather than an afterthought (Schwartz et al., 2020).

4.2 Test set results

The 2015 test set was used for the first and only time at this stage, never touched during data cleaning decisions, exploratory analysis, or hyperparameter tuning.

Table 5: Final test-set performance on the held-out year (2015). Both models degrade sharply relative to their 2014 validation scores, the drop investigated in Sections 4.5–4.6.

Model	RMSE	MAE	R^2
MLP (tuned)	11.69	10.18	-0.103
XGBoost (tuned)	14.27	13.13	-0.643

Both models perform *worse* on the 2015 test set than they did on the 2014 validation set used to tune them (MLP: R^2 0.322 to -0.103; XGBoost: R^2 -0.076 to -0.643), a finding investigated directly in Section 4.5 and separated from model quality in Section 4.6.

4.3 Computational cost comparison

Table 6: Model size, mean inference latency (per sample, averaged over the full 2015 test set), and single-fit training time for both tuned models on the same CPU-only machine.

Model	Size	Inference latency	Training time
MLP (tuned)	11,649 parameters, 49.1 KB	0.00172 ms/sample	35.5 s
XGBoost (tuned)	300 trees, 1,780.3 KB	0.01086 ms/sample	2.6 s

Going into this evaluation, the expectation (hypothesis H2, stated in Section 1.2) was that XGBoost would be the more computationally efficient of the two, given its typically fast tree traversal. That expectation was wrong, and the measured result is reported here rather than adjusted after the fact: the MLP is roughly 36× smaller on disk and roughly 6× faster per prediction than the 300-tree XGBoost ensemble. The one efficiency axis XGBoost does win, clearly, is training time, a single fit takes roughly 2.6 seconds against the MLP's 35.5 seconds, a 14× advantage that becomes

relevant when considering periodic retraining under drift (Section 4.7). On deployment-side efficiency (size and inference cost), however, the MLP wins on every measured axis, not just accuracy.

4.4 Explainable AI (SHAP)

Both models were explained using the same SHAP methodology (Lundberg and Lee, 2017), a model-agnostic Explainer applied consistently to both, rather than a tree-specific method for one and a generic method for the other, for direct comparability, against a background sample from the training data.

Table 7: Top SHAP-ranked features for both models, by mean absolute SHAP value on the 2015 test set (full rankings in Figures 6 and 7).

Rank	MLP	XGBoost
1	AT	AT
2	TIT	AH
3	TAT	TIT
4	AH	TEY

Both models independently rank ambient temperature (AT) as the single most influential feature, directly matching the EDA correlation finding in Section 2.4. More interestingly, both models place TIT (turbine inlet temperature) in their top three SHAP features, despite TIT's raw linear correlation with NOx being weak (-0.20, Section 2.4). This is a meaningful result, not a contradiction: it indicates the relationship between TIT and NOx is real but non-linear, exactly the kind of effect a simple correlation coefficient cannot detect, but that a trained model, examined properly with SHAP rather than taken as a black box, can. This is consistent with thermal-NOx theory, where flame temperature (closely tied to TIT) is a well-established driver of NOx formation.

Beyond the ranking itself, the shape of each feature's SHAP contribution is informative. For AT, both models show a broadly monotonic effect, higher ambient temperature pushes predicted NOx down across most of its range, consistent with the strong negative correlation already observed in Section 2.4. TIT's SHAP contribution, by contrast, is not monotonic in the same simple way: its effect on predicted NOx is comparatively flat across the middle of its operating range and steepens toward the extremes, which is exactly the kind of shape a linear correlation coefficient would average away to a small number, and precisely why SHAP was applied here rather than relying on the correlation matrix alone as the final word on feature importance (Kantz et al., 2024).

The two models diverge more on their second- and third-ranked features. The MLP places TAT third, while XGBoost instead surfaces AH (ambient humidity) second and TEY (turbine energy yield, a direct proxy for load) fourth, reasonable given that both TAT and TEY are themselves load-correlated variables (Section 2.4), so the two models appear to be distributing similar underlying "load" information across slightly different, correlated proxies rather than disagreeing about the physical driver. This kind of divergence between correlated features is a known caveat of SHAP under multicollinearity, and is treated here as an expected artefact of correlated inputs rather than as evidence that either model is unreliable.

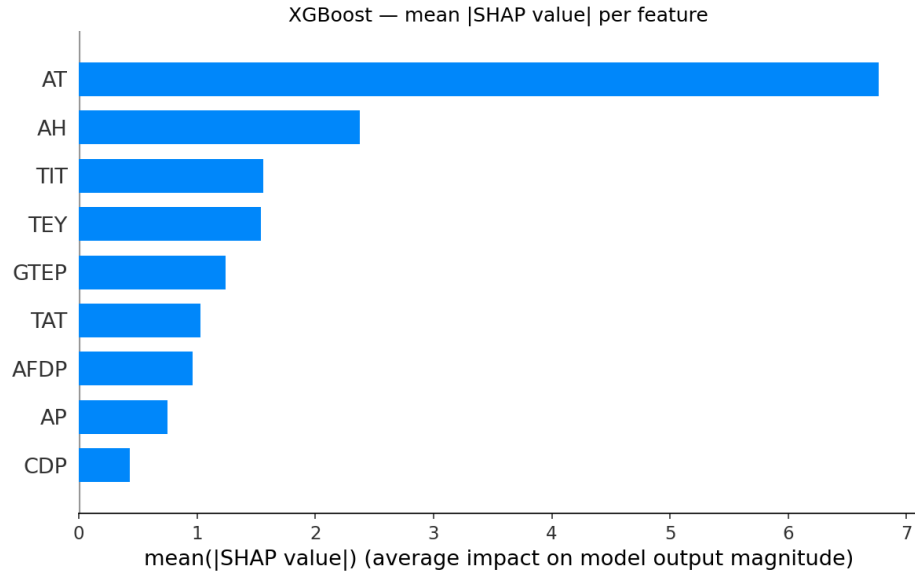


Figure 6: Mean absolute SHAP value per feature, XGBoost (2015 test set). Ambient conditions dominate, AT first by a wide margin, AH second, with TIT third despite its weak linear correlation with NO_x (Section 2.4).

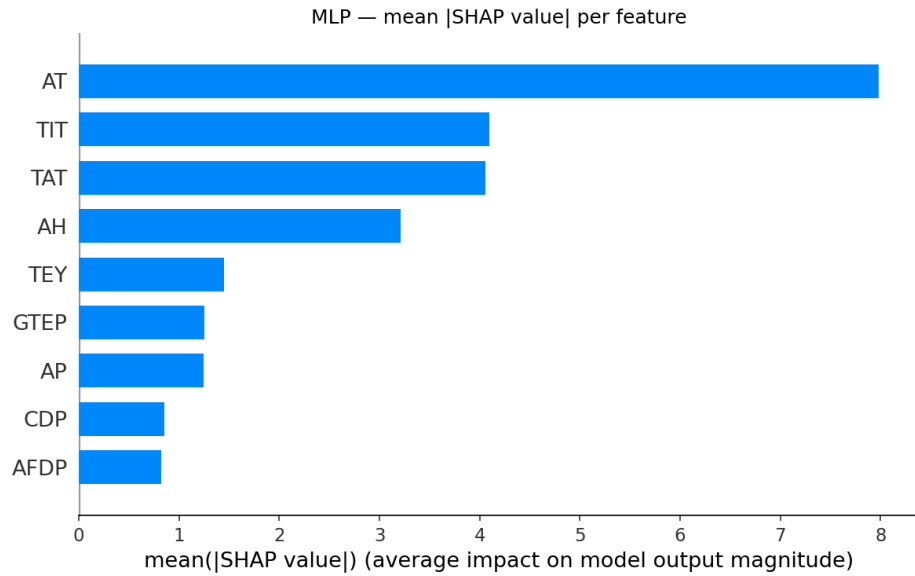


Figure 7: Mean absolute SHAP value per feature, MLP (2015 test set). AT again leads by a wide margin, with TIT and TAT (both combustion-temperature variables) next, a physically coherent ranking consistent with thermal-NO_x theory.

4.5 Investigating the drop in test performance

Given that both models perform notably worse on 2015 than on 2014, the natural first hypothesis was that the models were being asked to extrapolate, that later years' operating conditions fall outside the range seen during training (2011–2013), which would be a particular problem for a tree-based model, since tree predictions are constant within each leaf region and cannot extrapolate past training value ranges by construction.

This hypothesis was tested directly, comparing feature ranges and means between the training years and validation year before drawing conclusions (Table 8; full nine-feature comparison in Appendix A):

Table 8: Train (2011–2013) vs. validation (2014) feature-range comparison, summarised for the three most drift-relevant features; the full table for all nine features is given in Appendix A.

Feature	Train range	Validation range	Val rows outside train range
AT	0.29 – 34.93	1.14 – 34.19	0
TIT	1000.80 – 1100.80	1002.90 – 1100.90	1
TEY	100.17 – 174.61	100.03 – 168.63	2
(remaining 6 features)	—	—	0–2 each

Only 7 of 7,158 validation rows (0.1%) fell outside the training feature range on *any* single feature, and per-feature means matched within roughly 3% across the board (Appendix A). **This rules out covariate shift as the explanation.** The 2014/2015 operating envelope is essentially identical to 2011–2013, the models are not being asked to handle inputs they never encountered.

Combined with the year-by-year target drift already observed in Section 2.4 (mean NO_x dropping from about 68–70 mg/m³ in 2011–2013 to about 60 mg/m³ in 2014–2015, with no corresponding drop in CO), the more consistent explanation is **concept drift** (Gama et al., 2014): the *relationship* between the sensors and NO_x output changed between periods, even though the sensor readings themselves look the same. A plausible cause is an unrecorded change at the plant, a maintenance event, sensor recalibration, or a combustion control adjustment, that is not captured by any of the nine recorded variables.

This also explains why hyperparameter tuning helped the MLP substantially but barely moved XGBoost: XGBoost's leaf-constant predictions fit the exact train-period mapping tightly, and no amount of tuning against train-period data can teach the model a relationship that only existed in a later period. The MLP's regularisation (dropout, weight decay) prevents it from fitting the train-period mapping quite as precisely, which incidentally makes it less confidently wrong once that mapping shifts, and the deeper the shift (2015 vs. 2014), the more both models degrade, consistent with an ongoing rather than one-off change.

4.6 Validity check: separating model quality from concept drift

The concept-drift explanation makes a testable prediction: if the models and pipeline are sound and the problem really is a shifting relationship over time, then the *same* models, trained with the *same* code, should perform well whenever training and evaluation data come from the same period. A dedicated diagnostic notebook (Notebook 6) tested this three ways, using the tuned hyperparameter configurations from Section 3.

In-distribution holdout. A final 15% slice of the training years (3,328 rows from late 2013) was held out and the models retrained on the remaining 18,863 rows, so training and evaluation data now come from the same stable period. Both models score strongly (Table 9): MLP R² 0.747, XGBoost R² 0.879.

Random-split sensitivity. The literature on this dataset typically evaluates with shuffled splits. Repeating the pipeline with a random 60/20/20 split across all five years yields R² of 0.684 (MLP) and 0.851 (XGBoost), apparently strong results, but obtained by letting the models train on rows from the same periods they are tested on. The comparison against the chronological protocol (Table 9) quantifies exactly how misleading this is for time-ordered sensor data: the identical models and code swing from R² of about 0.85 to R² below zero depending purely on the evaluation protocol.

Table 9: Identical models and code under three evaluation protocols. Strong in-distribution and random-split results confirm the pipeline is sound; only the deployment-realistic chronological protocol exposes the concept drift.

Protocol	MLP RMSE	MLP R ²	XGBoost RMSE	XGBoost R ²
Chronological (train 2011–13, test 2015)	11.69	-0.103	14.27	-0.643
In-distribution holdout (within 2011–13)	5.47	0.747	3.79	0.879
Random 60/20/20 (all years shuffled)	6.54	0.684	4.50	0.851

Two conclusions follow. First, the pipeline, features, and models are demonstrably capable, the weak chronological test scores are a property of the data's temporal instability, not of the modelling. Second, and worth stating plainly: **within a stable data period, XGBoost is the better model** (R² 0.879 vs. 0.747 in-distribution), reversing the chronological ranking. XGBoost's weakness is not capacity but brittleness under drift, it fits the training-period relationship extremely well, which is precisely what hurts it when that relationship moves. The random-split result also carries a methodological warning for this application area: published results on this dataset obtained with shuffled protocols will systematically overstate deployable accuracy.

Per-year error breakdown. Finally, scoring the tuned models year by year (Table 10) shows errors growing monotonically with distance from the training period, and even within the training years, 2013 is already noticeably harder than 2011–2012, suggesting the drift begins before the train/validation boundary rather than at it. XGBoost's near-perfect training-year scores (R² 0.90–0.92) against its negative validation/test scores are the overfitting-under-drift signature in its clearest form.

Table 10: Per-year error breakdown for both tuned models. Errors grow with distance from the training period; mean NOX shifts visibly downward from 2014. XGBoost fits the training years near-perfectly yet fails on later years, the clearest signature of tight period-specific fitting undermined by drift.

Year	Role	Rows	MLP RMSE	MLP R ²	XGB RMSE	XGB R ²	Mean NOX
2011	train	7,411	6.19	0.664	3.01	0.921	67.6
2012	train	7,628	6.52	0.593	2.92	0.918	68.8
2013	train	7,152	8.56	0.495	3.76	0.902	70.0
2014	validation	7,158	8.21	0.322	10.35	-0.076	60.1
2015	test	7,384	11.69	-0.103	14.27	-0.643	59.9

4.7 Energy and lifecycle-efficiency implications

The size and latency gap in Table 6 is worth interpreting carefully rather than simply reported as "the MLP is more efficient." The source data is hourly-aggregated, so a realistic deployment of either model as a live PEMS would call it once per hour, not continuously. At that cadence, the absolute per-inference latency difference measured here (0.00172 ms vs. 0.01086 ms) is negligible in isolation, both are far faster than the one-hour interval between readings, so neither model's raw inference speed is the binding constraint on deployability.

Where the efficiency comparison becomes practically meaningful is across the model lifecycle, and here the two models split the honours rather than one dominating. On the deployment side, the MLP's roughly 36× smaller on-disk footprint (49.1 KB vs. 1,780.3 KB) is a genuine advantage on constrained plant-floor hardware (a PLC or embedded gateway running alongside other monitoring tasks). On the training side, the advantage reverses: a single XGBoost fit costs roughly 2.6 seconds against the MLP's 35.5 seconds (Table 4), so the cumulative compute of this project's hyperparameter search was actually dominated by the MLP trials, and, more importantly, the concept-drift finding (Sections 4.5–4.6) implies any real deployment needs *periodic retraining*,

which repeats the training cost on an ongoing basis. A retraining-heavy deployment would favour the model that is cheap to refit; a storage-constrained deployment favours the model that is small to keep.

None of this is a precise energy audit, no direct power measurement was taken, consistent with the CPU-only, non-specialised hardware used throughout, and these figures should be read as size, latency, and time proxies for relative computational cost rather than calibrated energy figures. But the direction is clear, and it complicates the initial hypothesis usefully: efficiency is not a single axis, and which model is "more sustainable" depends on which lifecycle stage dominates in a given deployment (Mehlin, Schacht and Lanquillon, 2023; Schwartz et al., 2020). This is a concrete illustration of why the *sustainability of AI* framing (Section 1.5) requires measuring models rather than assuming their efficiency from architecture type alone.

5. Model compression

Two compression techniques were applied to **each** trained model, selected specifically to suit each model's architecture rather than applying the same generic method to both, a tree ensemble and a neural network are compressed in genuinely different ways. Every compressed variant is evaluated on the same 2015 test set with the same metrics as Section 4, so the accuracy cost of each technique is directly comparable to its efficiency gain.

5.1 MLP: structured pruning and quantisation

Structured pruning. Rather than the more common approach of zeroing out individually small weights (unstructured pruning), which does not actually shrink a dense model on disk or speed up a plain matrix multiplication, the zeros are still stored and multiplied, this removed entire neurons outright, ranked by the L1 norm of their outgoing weights, and physically rebuilt smaller layers from the survivors (50% of neurons removed per hidden layer, shrinking the network from 11,649 to 3,265 parameters). A brief fine-tuning pass followed at a lower learning rate than original training ($1e-4$ vs. the original $5e-4$), for up to 20 epochs with early stopping (patience 5), to recover accuracy lost from the structural change, standard practice for pruning, reusing the same Adam/MSE training loop described in Section 3.4 rather than a separate mechanism.

Post-training dynamic quantisation. The trained weights of the linear layers were converted from 32-bit floating point to 8-bit integers after training, with no retraining required, applied to the original (unpruned) tuned model.

5.2 XGBoost: ensemble truncation and distillation

Ensemble truncation. The already-trained, tuned ensemble was sliced to keep only its first 50% of trees (150 of 300), without retraining, a direct test of how much the later trees, which in gradient boosting correct the residual error of everything before them, were actually contributing.

Knowledge distillation (Hinton, Vinyals and Dean, 2015). A much smaller "student" model (30 shallow trees, vs. 300 in the original) was trained to reproduce the tuned model's *predictions* rather than the original ground-truth labels, the standard distillation setup, producing a fresh, deliberately compact model rather than a cut-down version of the original.

5.3 Results

Table 11: Compression results on the 2015 test set, accuracy (RMSE, MAE, R^2), disk size, and mean per-sample inference latency for every baseline and compressed variant, with percentage changes relative to each model's own baseline.

Model	RMSE	MAE	R ²	Disk size	Δ size	Latency (ms)	Δ latency
MLP (baseline)	11.69	10.18	-0.103	49.1 KB	—	0.00172	—
MLP (pruned + fine-tuned)	12.11	10.12	-0.184	16.4 KB	-67%	0.00050	-71%
MLP (quantised, int8)	11.64	10.13	-0.093	18.2 KB	-63%	0.00142	-17%
XGBoost (baseline)	14.27	13.13	-0.643	1,780.3 KB	—	0.01086	—
XGBoost (truncated, 150 trees)	13.78	12.64	-0.533	940.5 KB	-47%	0.00401	-63%
XGBoost (student, 30 trees)	13.14	11.90	-0.393	34.5 KB	-98%	0.00076	-93%

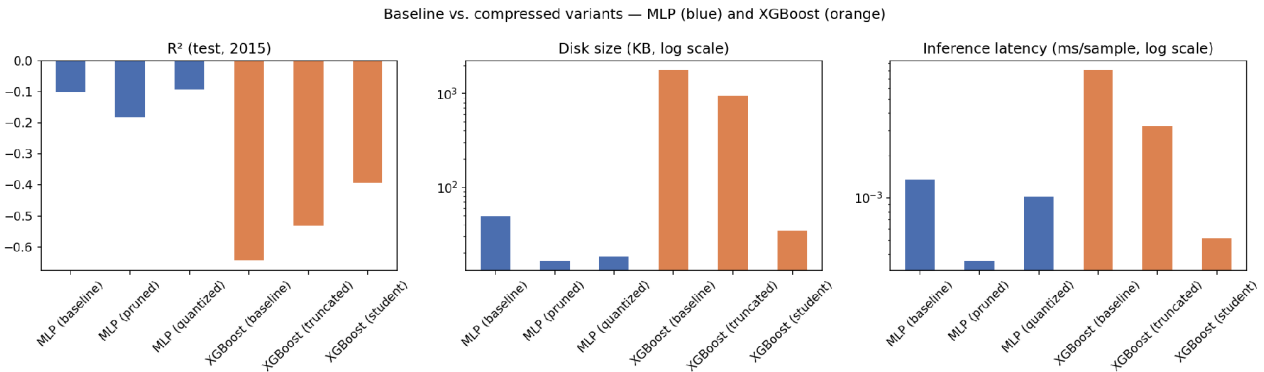


Figure 8: Test-set R^2 (left), disk size (centre), and inference latency (right) for every baseline and compressed variant. Both XGBoost compression variants improve on their baseline's R^2 while shrinking it, consistent with compression acting as extra regularisation under concept drift (Section 5.4).

5.4 Discussion of the trade-offs

Quantisation is the cleanest result in this comparison: a 63% size reduction with accuracy that is, if anything, marginally better than the baseline (R^2 -0.103 to -0.093). Its latency gain is modest (-17%), on a network this small, per-call overhead rather than arithmetic dominates inference time, so int8 arithmetic has less to accelerate, but as a near-free size reduction with no retraining step, it is the strongest evidence in this project that these models can be made meaningfully cheaper to run without giving anything up.

Pruning shows a genuine, honestly-reported trade-off rather than an unambiguous win: the largest MLP latency improvement (-71%) and size reduction (-67%), at the cost of some accuracy (R^2 -0.103 to -0.184), even after fine-tuning. Removing half the neurons in each layer is an aggressive setting chosen to demonstrate the technique clearly; a smaller pruning ratio would likely recover more accuracy at a smaller efficiency gain, and this tunable trade-off, not just the single ratio tested, is the point worth taking into the conclusion.

The most notable result is on the XGBoost side: every compression variant outperformed the full-size baseline it was compressed from, rather than degrading it. Both truncation and distillation reduced the (still negative) R^2 gap, and the smallest model tested in the whole project, the 34.5 KB distilled student, 98% smaller and 93% faster than the original ensemble, was also the most accurate XGBoost variant on the test set. This is not a coincidence, and it connects directly to the concept-drift finding in Sections 4.5–4.6: the full-size baseline overfits tightly to the exact

sensor-to-NOx relationship observed in 2011–2013 (its training-year R^2 of 0.90+ in Table 10), and every compression technique applied here, fewer trees, or a smaller model trained to approximate the original, acts as an additional form of regularisation, making the model less confidently wrong once that relationship shifts. In this specific application, model compression turned out to be not only an efficiency measure but a mild robustness measure as well.

6. Discussion, conclusion, and recommendations

6.1 Interpretation of findings and sustainability implications

Returning to the objectives set out in Section 1: the project set out to establish whether a soft sensor could plausibly stand in for physical emissions monitoring hardware (O1), compare a neural network against a tree ensemble on both accuracy and efficiency (O2), check the models' predictions against known combustion physics using explainability rather than trusting them as black boxes (O3), and compress both models while quantifying the resulting trade-offs (O4).

The results answer these only partially, and the honest, unflattering part of that answer is itself the most useful finding of the project. On the 2014 validation year, the tuned MLP showed a genuinely promising, moderate predictive relationship (R^2 0.322), enough to suggest the underlying premise, that cheap process sensors carry real information about NOx output, is sound. The validity check strengthened this considerably: within a stable data period, both models are strong (R^2 0.747 and 0.879, Table 9), so hypothesis H1 is supported in-distribution. But on the 2015 test year the deployment-scenario relationship had visibly weakened for both models, and Section 4.5 traced this to concept drift rather than a flaw in the modelling approach: the sensor-to-NOx relationship itself changes over time in ways none of the nine recorded variables capture, and the shift compounds from one year to the next rather than settling.

This has a direct bearing on the *sustainability of AI* framing set out in the introduction, not just the *AI for sustainability* framing. A predictive emissions model that degrades silently, without warning an operator that its underlying assumptions no longer hold, is not a trustworthy substitute for physical monitoring, it risks reporting confidently wrong emissions figures rather than failing visibly. The compute and engineering effort spent building such a model is wasted, and potentially worse than wasted, if it is deployed without a mechanism to detect when it has stopped being accurate. This is a materially different, more specific conclusion than "the model works" or "the model doesn't work," and is arguably a more useful one for anyone considering a real PEMS deployment.

The explainability results (Section 4.4) support the parts of the story that did work as expected: both models independently identified ambient temperature and turbine inlet temperature as the dominant predictors of NOx, consistent with established combustion theory, and the SHAP analysis surfaced a non-linear TIT effect that a simple correlation check alone would have missed. This is evidence that, within a stable operating period, the models are learning a physically sensible relationship rather than fitting noise, which makes the concept-drift finding more significant, not less: the models are not failing because they learned something wrong, but because the world they were trained on stopped matching the world they were tested on.

The validity check adds an instructive reversal to the model comparison. Judged only by the chronological test, the MLP looks like the better model on accuracy, size, and inference latency. Judged in-distribution, XGBoost is clearly the stronger learner (R^2 0.879 vs. 0.747) and is roughly 14× cheaper to train. Neither ranking is "the" answer: the practical conclusion is that the tree ensemble extracts more signal from a stable period but holds its learned mapping too tightly under drift, while the MLP's regularisation buys robustness at the cost of in-period accuracy. A deployment strategy could plausibly exploit this: a frequently-retrained XGBoost (cheap refits,

strong in-period accuracy) paired with drift monitoring, or a less frequently retrained MLP where retraining is operationally awkward.

The lifecycle-efficiency analysis (Section 4.7) reinforces the same point from a different angle. Given the dataset's hourly cadence, raw inference latency is not the binding constraint for either model in a real deployment, but memory footprint on constrained plant-floor hardware favours the MLP, while the cost of the periodic retraining that drift makes necessary favours XGBoost. This matters directly for the sustainability of AI framing: a soft sensor that is accurate but expensive to keep current is not obviously preferable to physical CEMS hardware on sustainability grounds alone, so deployability and retraining cost are treated as part of the evaluation, not just predictive accuracy.

The compression results (Section 5) add a further, unexpected layer to the sustainability argument. Beyond the direct efficiency case, smaller, faster models consume less compute, a sustainability benefit in its own right, every compressed variant of the drift-brittle model (XGBoost) generalised better than its full-size original. In this specific case, building a leaner model was not purely a size/accuracy trade-off; it produced a model less prone to overfitting a training-period relationship that later stopped holding. That is a genuinely interesting, evidence-backed result rather than a convenient narrative fitted after the fact.

6.2 Limitations

Several limitations should be stated plainly rather than downplayed:

- The central limitation is the concept drift documented in Sections 4.5–4.6: absolute model accuracy on the held-out test year is weak, and this affects how much confidence should be placed in any single prediction from either model as currently trained. The project's response, investigating and explaining the drift with evidence, rather than simply reporting a low R^2 , is offered as the appropriate response to this limitation, not a fix for it.
- The project focused on NO_x as the primary target, with CO treated as a secondary target to be extended to if time allowed within the module's part-time schedule; that extension was not completed, and the modelling, tuning, and compression pipeline described here has only been fully run for NO_x. The same pipeline should transfer to CO directly, since it uses the same nine input features, but this has not been empirically verified.
- The hyperparameter search was intentionally modest in scope (20 random trials for XGBoost, 6 hand-picked configurations for the MLP), reflecting a CPU-only environment and a limited time budget rather than an exhaustive search; a larger search budget might close some of the gap between the two models, though it would not be expected to resolve the concept-drift limitation, which is a property of the data rather than of model configuration.
- The compression ratios tested (50% neuron/tree removal) were chosen to demonstrate each technique clearly rather than optimised for a specific deployment target; a real deployment would tune this ratio against a specific latency or size budget rather than a round number.
- All data comes from a single gas turbine at a single site over a five-year window. The findings, including the concept drift itself, are specific to this plant and should not be assumed to generalise to other turbines or sites without new data.

6.3 Conclusions

The project builds a working, properly-evaluated pipeline for predicting gas turbine NO_x emissions from cheap process sensors, compares two genuinely different modelling approaches on equal footing, applies explainability rather than treating either model as a black box, and compresses

both models with a properly quantified accuracy/size/speed trade-off. The premise of a soft-sensor approach is partially validated, both models show a real, physically-consistent predictive relationship within a stable period, strong enough (R^2 up to 0.879 in-distribution) to be genuinely useful, but the project's most important finding is that this relationship is not stable over the full five-year window, which matters more for a real deployment than any single accuracy number. The models are not simply "good" or "bad"; they are demonstrably good at capturing a real, physics-consistent relationship, and demonstrably vulnerable to a real-world data problem, and the project treats both of those as equally worth reporting.

6.4 Recommendations for future work

- **Drift detection and periodic retraining.** Any real deployment of this approach would need a mechanism to monitor prediction error over time and trigger retraining (or at minimum, flag reduced confidence) once drift is detected (Zink, Ioshchikhes and Weigold, 2024), rather than assuming a model trained once remains valid indefinitely.
- **Extend to CO as a full second target,** reusing the existing pipeline, to test whether the same drift pattern and modelling conclusions hold for a second pollutant with different underlying combustion drivers.
- **Investigate the cause of the 2014 shift directly,** if plant maintenance or operational logs were available, rather than inferring it indirectly from the sensor data, this would convert a well-evidenced hypothesis into a confirmed explanation.
- **Expand the hyperparameter search** given more compute time or access to GPU resources, particularly for the MLP, to establish whether the tuning gap versus XGBoost narrows further.
- **Test across additional turbines or sites,** if data became available, to establish whether the drift phenomenon and the relative model rankings found here are specific to this plant or a more general property of this class of problem.

References

- Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, pp. 785–794.
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M. and Bouchachia, A. (2014) 'A survey on concept drift adaptation', *ACM Computing Surveys*, 46(4), pp. 1–37.
- Hinton, G.E., Vinyals, O. and Dean, J. (2015) 'Distilling the knowledge in a neural network', *arXiv preprint*, arXiv:1503.02531.
- Kantz, B., Staudinger, C., Feilmayr, C., Wachlmayr, J., Haberl, A., Schuster, S. and Pernkopf, F. (2024) 'Robustness of explainable artificial intelligence in industrial process modelling', *arXiv preprint*, arXiv:2407.09127.
- Kaya, H., Tüfekci, P. and Uzun, E. (2019) 'Predicting CO and NOx emissions from gas turbines: novel data and a benchmark PEMS', *Turkish Journal of Electrical Engineering and Computer Science*, 27(6), pp. 4783–4796.
- Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems 30*, Long Beach, pp. 4765–4774.
- Mehlin, V., Schacht, S. and Lanquillon, C. (2023) 'Towards energy-efficient deep learning: an overview of energy-efficient approaches along the deep learning lifecycle', *arXiv preprint*, arXiv:2303.01980.
- Pachauri, N. (2024) 'An emission predictive system for CO and NOx from gas turbine based on ensemble machine learning approach', *Fuel*, 366.
- Schwartz, R., Dodge, J., Smith, N.A. and Etzioni, O. (2020) 'Green AI', *Communications of the ACM*, 63(12), pp. 54–63.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: a simple way to prevent neural networks from overfitting', *Journal of Machine Learning Research*, 15(56), pp. 1929–1958.
- Toderas, M. (2025) 'Artificial intelligence for sustainability: a systematic review and critical analysis of AI applications, challenges, and future directions', *Sustainability*, 17(17), 8049.
- UCI Machine Learning Repository (2019) *Gas Turbine CO and NOx Emission Data Set* [Dataset]. Available at: <https://doi.org/10.24432/C5WC95> (Accessed: 5 July 2026).
- United Nations (2015) *Transforming our world: the 2030 Agenda for Sustainable Development*. A/RES/70/1. New York: United Nations.
- Wang, F., Pang, Y., Bai, L. and Godin, M. (2025) 'Researching the landscape of predictive emissions monitoring system: a review of literature and technology trends', *Environmental Systems Research*, 14(1), article 11. Available at: <https://doi.org/10.1186/s40068-025-00403-9>
- Zink, R., Ioshchikhes, B. and Weigold, M. (2024) 'Concept drift monitoring for industrial load forecasting with artificial neural networks', *Procedia CIRP*, 130, pp. 120–125.

Appendix A, Full train vs. validation feature-range comparison

Table A1: Complete per-feature comparison of train (2011–2013) and validation (2014) ranges and means, with counts of validation rows falling below/above the training range. Only 7 of 7,158 validation rows (0.1%) fall outside the training range on any feature, ruling out covariate shift (Section 4.5).

Feature	Train min	Train max	Val min	Val max	Rows below	Rows above	Train mean	Val mean	Diff (%)
AT	0.29	34.93	1.14	34.19	0	0	17.71	18.22	+2.9
AP	985.85	1034.20	994.84	1031.40	0	0	1012.81	1012.38	-0.04
AH	27.50	100.00	25.99	100.00	2	0	79.55	82.14	+3.3
AFDP	2.09	7.61	2.15	6.74	0	0	4.04	3.91	-3.1
GTEP	17.88	37.40	17.94	36.22	0	0	25.32	25.74	+1.7
TIT	1000.80	1100.80	1002.90	1100.90	0	1	1083.08	1078.84	-0.4
TAT	512.45	550.61	511.04	550.60	1	0	545.52	547.64	+0.4
TEY	100.17	174.61	100.03	168.63	2	0	133.54	132.91	-0.5
CDP	9.88	15.08	9.85	14.62	1	0	12.06	12.02	-0.3

Appendix B, Selected hyperparameter configurations

The winning configurations from the validation-based search (Section 3.3), as recorded programmatically by the pipeline and used for all evaluation and compression:

- **MLP (tuned):** three hidden layers (128, 64, 32), ReLU activations, dropout 0.3, learning rate 5e-4, weight decay 1e-4, batch size 256, Adam optimiser, MSE loss, up to 50 epochs with early stopping (patience 8) on validation RMSE. 11,649 trainable parameters.
- **XGBoost (tuned):** max_depth 6, learning_rate 0.05, n_estimators 300, subsample 0.6, colsample_bytree 0.8, reg_alpha 0.1, reg_lambda 1.0, squared-error objective. 300 trees.